

Course Title: Algorithms Lab

Course Code: CSE-2202

Experiment Title: Merge Sort

Submitted by: Shafin Mehrab
ID:251014058

Submitted to: Fatema Khan
Lecturer, University of Liberal Arts and Bangladesh

Date of Submission: October 9, 2026

Contents

1 Objectives **1**

2 Problem Statement and Requirements **1**

3 Algorithm Design **2**
3.1 Approach 2
3.2 Data Structures 3
3.3 Pseudocode 4
3.4 Worked Trace 6

4 Complexity Analysis **6**

5 Implementation **7**
5.1 Output 7

6 Discussion **8**

7 Conclusion **8**

8 References **8**

1 Objectives

1. **Implement** the Merge Sort algorithm to sort an array of characters into ascending order, while rejecting vowel characters during input.
2. **Trace** the merge operations to demonstrate how single-element subarrays are combined sequentially into larger sorted subarrays.
3. **Analyze** the time complexity of Merge Sort in the best, average, and worst cases, and compare it with the observed behaviour.

2 Problem Statement and Requirements

The task requires writing a program that reads an array of characters from the user and sorts them in ascending order using the Merge Sort algorithm. The program must enforce the following constraints:

- The user enters the number of elements, n, where $1 \leq n \leq 100$.
- The user must enter exactly n characters.
- Vowels (a, e, i, o, u in both uppercase and lowercase) are **not allowed**. If a vowel is entered, the program must reject it and ask the user to enter another character.
- Characters are compared by their character codes (ASCII values), so uppercase and lowercase letters are treated as distinct.

The program must display:

1. The original array.
2. Each merging step performed by Merge Sort, showing:
   (a) The two subarrays being merged.
   (b) The resulting sorted subarray after merging.
3. The final sorted array in ascending order.

Input, Output, and Constraints

Item	Description
Input	• n: integer, $1 \leq n \leq 100$ • n characters: each must be a non-vowel character • Invalid input (vowels) must be rejected and re-prompted
Output	• Original array • Each merge step (subarrays and result) • Final sorted array in ascending order
Constraints	• $1 \leq n \leq 100$ • Vowels are forbidden • Characters are compared by ASCII codes • Uppercase and lowercase are distinct
Assumptions	• Input characters are printable ASCII characters • No duplicate handling is required
Special Cases	• $n = 1$: no merge, output is the single character • All characters already sorted: still performs merges • Repeated vowel input: program keeps prompting until valid character is entered

Table 1: Input, output, and constraints for Task 2.

3 Algorithm Design

3.1 Approach

Merge Sort is based on Divide and Conquer algorithm. The idea is to:

1. **Divide:** Split the array into two halves recursively until each subarray contains a single element.
2. **Conquer:** Merge the subarrays back together in sorted order.

3. **Combine:** At each merge step, compare the smallest remaining elements of the two subarrays and place the smaller one into the result array.

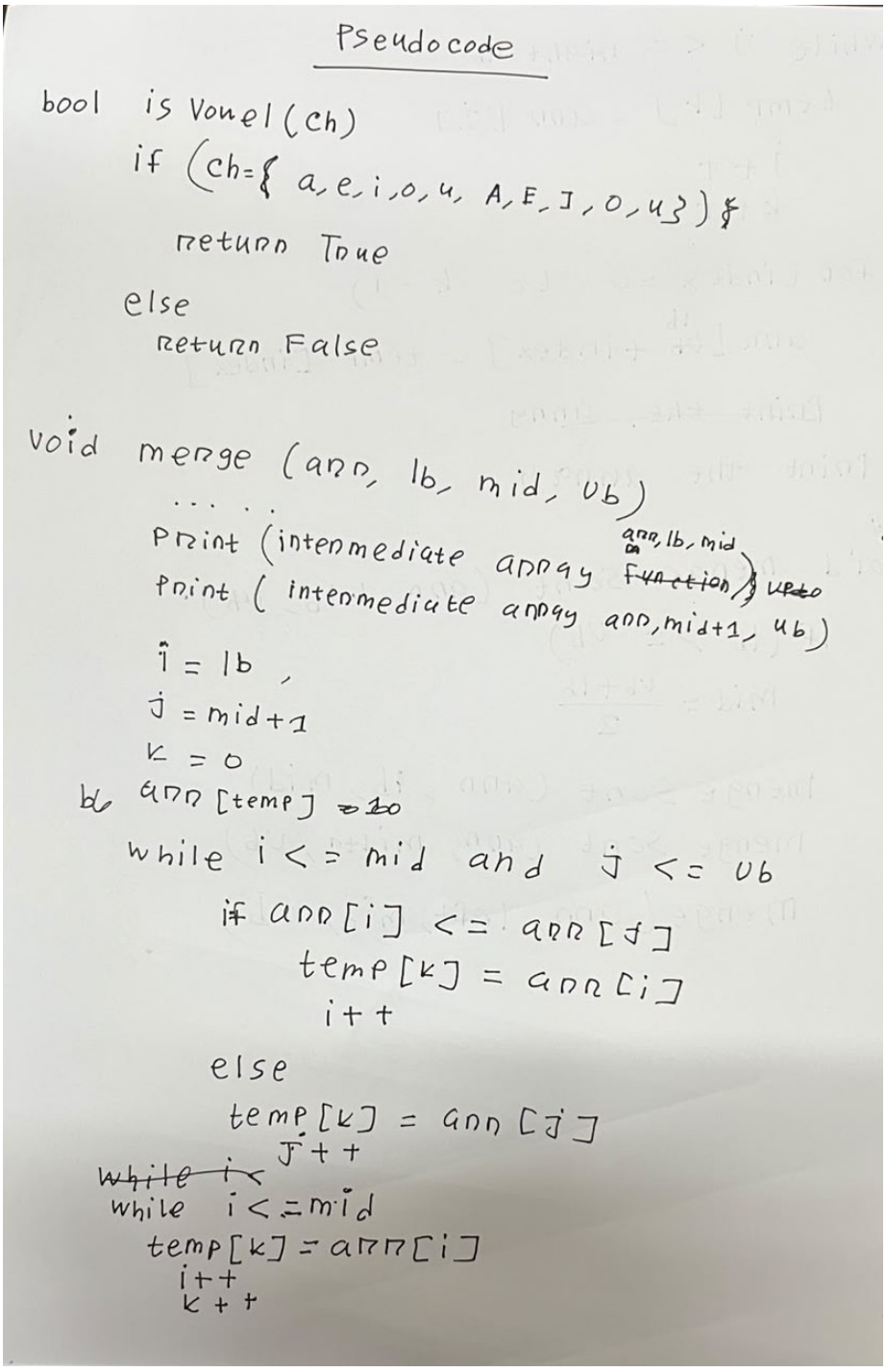
The algorithm is suitable here because:

- It guarantees $O(n \log n)$ time complexity in all cases.
- It is stable and works well for small to moderate input sizes ($n \leq 100$).
- The merge steps are naturally visualizable, which matches the requirement to display each merging operation.

3.2 Data Structures

- **Array of characters:** Stores the input and intermediate sorted subarrays.
- **Temporary array:** Used during merging to hold the combined sorted result before copying back.
- **Recursion stack:** Implicitly used by the recursive merge sort function.

3.3 Pseudocode



Pseudocode

```
bool isVowel(ch)
    if (ch={a,e,i,o,u,A,E,I,O,U}){
        return True
    else
        return False

void merge(arr, lb, mid, ub)
    ....
    Print(intermediate array arr,lb,mid Function) upto
    Print(intermediate array arr,mid+1, ub)
    i = lb
    j = mid+1
    k = 0
    b[ arr[temp] = 100
    while i <= mid and j <= ub
        if arr[i] <= arr[j]
            temp[k] = arr[i]
            i++
        else
            temp[k] = arr[j]
            j++
    while i <= mid
        temp[k] = arr[i]
        i++
        k++
```

Figure 1: Handwritten pseudocode, part 1.

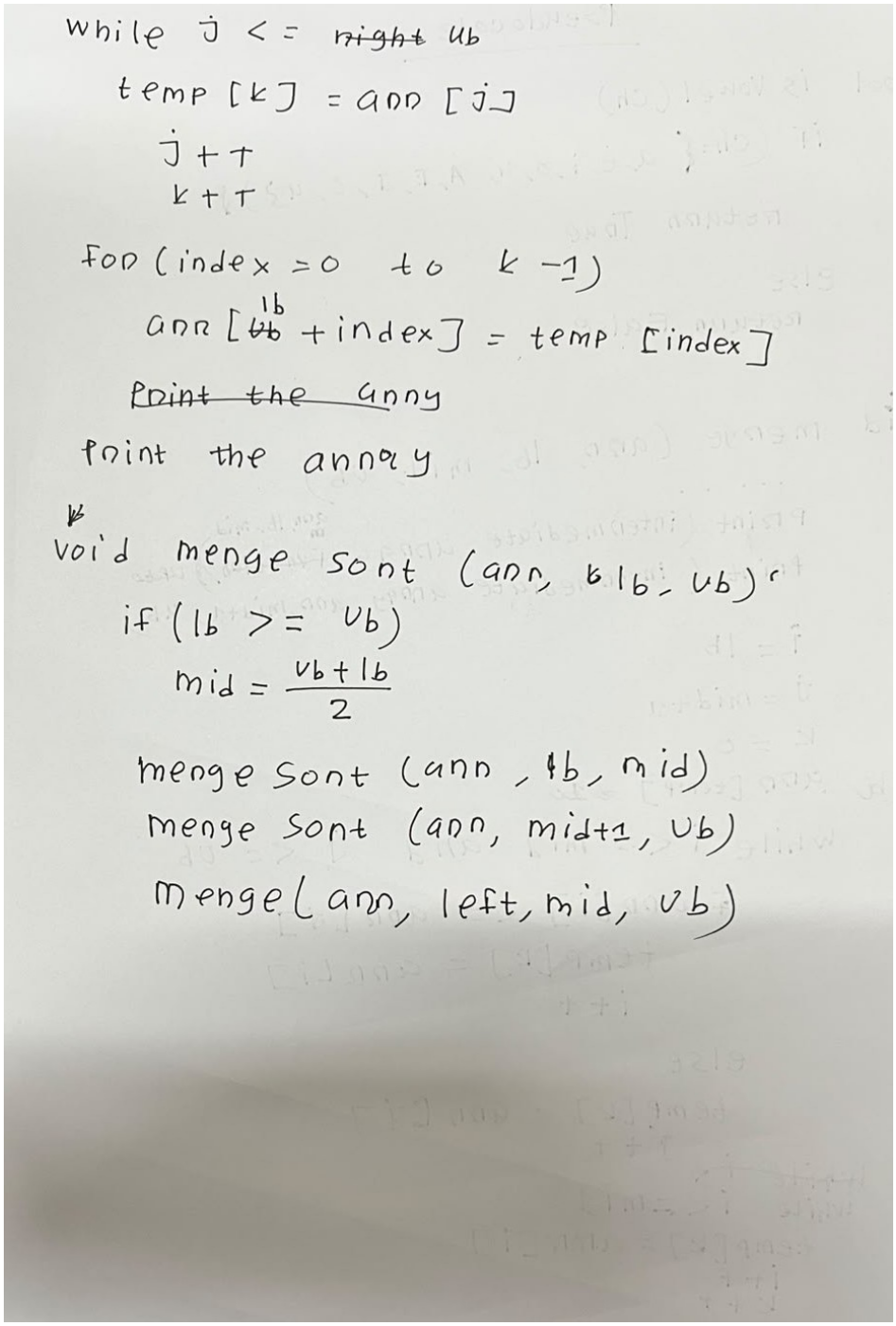


Figure 2: Handwritten pseudocode, part 2.

3.4 Worked Trace

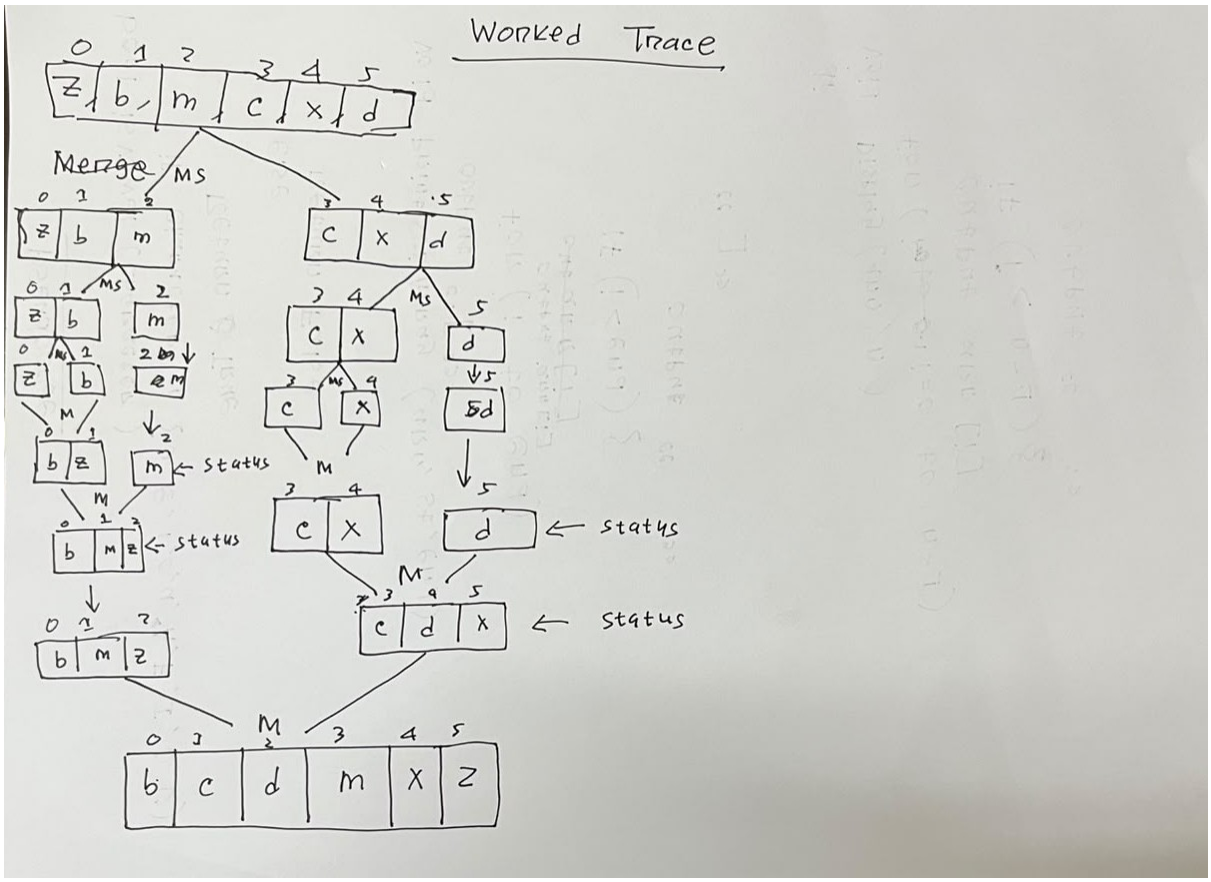


Figure 3: Worked trace of Merge Sort for the input array `[z, b, m, c, x, d]`.

4 Complexity Analysis

Let n be the number of elements in the array.

Recurrence Relation

$$T(n) = \begin{cases} O(1) & \text{if } n \leq 1 \\ 2T(n/2) + O(n) & \text{if } n > 1 \end{cases}$$

The $O(n)$ term comes from the merge operation, which compares and copies each element once per level of recursion.

Time Complexity

Case	Reason	Time Complexity
Best	Input is already sorted; merge still processes all elements at each level	$O(n \log n)$
Average	Random input; same number of levels and merge operations	$O(n \log n)$
Worst	Reverse sorted input; same number of levels and merge operations	$O(n \log n)$

Table 2: Time complexity of Merge Sort.

5 Implementation

The essential functions are:

- `bool isVowel(char c)`: Returns `true` if the character is a vowel (case-insensitive).
- `void merge(char arr[], int left, int mid, int right)`: Merges two sorted subarrays and displays the merge step.
- `void mergeSort(char arr[], int left, int right)`: Recursively divides and sorts the array.
- `void printArray(char arr[], int n)`: Prints the array elements.

5.1 Output

```
PS C:\Users\shafi\OneDrive\Documents\Tutorial project\Lab Tasks\1\2> cd "c:\Users\shafi\OneDrive\Documents\Tutorial project\Lab Tasks\Algo lab\2\" ; if ($?) { g++ Lab_Task_2.cpp -o Lab_Task_2 } ; if ($
?) { .\Lab_Task_2 }
Enter the number of elements (1-100): 6
Enter 6 non-vowel characters:
z
b
m
c
x
d

Original array: z b m c x d

Merge Sort Steps
Merging: [z] + [b]
Result: [b z]
Merging: [b z] + [m]
Result: [b m z]
Merging: [c] + [x]
Result: [c x]
Merging: [c x] + [d]
Result: [c d x]
Merging: [b m z] + [c d x]
Result: [b c d m x z]

Final sorted array: b c d m x z
```

Figure 4: Successful Merge Sort run, including the merge steps and final sorted array.

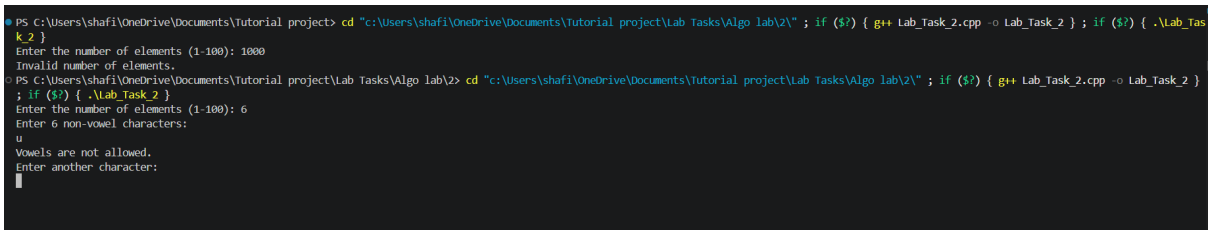


Figure 5: Input validation examples for an invalid element count and vowel rejection.

6 Discussion

The implementation successfully met all requirements:

- The program correctly reads n characters, rejects vowels, and re-prompts until valid characters are entered.
- Merge Sort is implemented recursively and displays each merge step, clearly showing how single-element subarrays are combined.
- The final sorted array is displayed in ascending order based on ASCII values.

The theoretical time complexity of Merge Sort is $O(n \log n)$ in all cases. For $n = 6$, the number of comparisons and merge operations is small, and the observed output matches the expected trace. The vowel rejection logic adds a constant-time check per input character, which does not affect the overall complexity.

One limitation is that the program uses additional memory for temporary arrays during merging, resulting in $O(n)$ auxiliary space. For $n \leq 100$, this is negligible. Another consideration is that the program treats uppercase and lowercase letters as distinct, which is consistent with the problem statement.

7 Conclusion

In this experiment, I implemented a C++ program that reads an array of non-vowel characters from the user and sorts them in ascending order using the Merge Sort algorithm. The program successfully rejects vowels, displays each merge step, and outputs the final sorted array. The time complexity of Merge Sort is $O(n \log n)$ in the best, average, and worst cases, supported by the recurrence $T(n) = 2T(n/2) + O(n)$. A key lesson is that Divide-and-Conquer algorithms like Merge Sort provide predictable performance and allows to step-by-step visualise, which is valuable for understanding.

8 References

No external resources were used.